

Chapter 1

Introduction

Over the past few years, the way people access everyday services has changed significantly due to the availability of digital platforms. Booking food, transport, or shopping online has become routine. However, when it comes to basic local services such as plumbing, electrical repair, cleaning, or carpentry, the situation is very different, especially in semi-urban cities like Nanded. Even today, most people depend on phone numbers saved in their contacts, local shop references, or word-of-mouth suggestions to find service providers.

From our own experience in areas such as Vishnupuri, Degloor Naka, and Bhagyanagar, arranging a simple service often involves multiple phone calls, uncertain availability, and no clarity about pricing or service reliability. During urgent situations, such as water leakage or power failure, this lack of a structured system becomes even more problematic. Customers have no way to track requests, and service providers have no organized method to manage their work.

At the same time, many skilled local professionals struggle to find regular work. Most depend on informal networks, local contractors, or personal contacts. If a provider is new or lacks strong connections, getting consistent work becomes difficult, regardless of skill level. This gap between customer needs and service provider accessibility highlights a clear problem in the local service ecosystem.

We observed that while large on-demand service platforms exist, they mainly focus on metropolitan cities and are often unsuitable for smaller regions like Nanded. These platforms either do not operate locally or fail to adapt to regional constraints such as limited digital literacy and hyperlocal service demand. This observation formed the foundation for the MH26 Services platform.

1.1 Purpose and Inspiration

The purpose of the MH26 Services platform is to organize and digitize local service coordination in a way that is practical, simple, and relevant to smaller cities. We aimed to create a system that reduces the dependency on phone calls, personal

references, and informal communication by bringing customers and service providers onto a single platform.

The inspiration for this project came directly from real-life situations we encountered. In many households, arranging services such as plumbing or electrical repair requires calling multiple people, waiting for responses, and often settling for whoever is available rather than who is reliable. We noticed that there is no straightforward way to verify service providers or track whether a service request is being handled properly.

At the same time, we observed that many local service providers are skilled but lack digital visibility. They depend heavily on daily availability, personal connections, or contractors for work. We wanted to design a system that gives these professionals fair access to customers while also ensuring trust and accountability through a structured onboarding and verification process.

Instead of designing a generic solution meant for all cities, we intentionally focused on a specific region. By limiting the scope to a local environment, we were able to account for practical factors such as varying levels of digital familiarity and localized service demand. This approach helped us keep the system realistic, usable, and aligned with real-world conditions.

Overall, the purpose of MH26 Services is not just to build another service platform, but to demonstrate how technology can be applied at a community level to solve everyday coordination problems in a structured and reliable manner.

1.2 Objective

The MH26 Services project was developed with clear and practical objectives that guided both the design and implementation of the system. Rather than focusing on abstract goals, these objectives were framed around real problems observed during daily service coordination.

The primary objective is to provide a centralized digital platform where users can easily discover and book local services without relying on informal communication methods. Another important goal is to reduce uncertainty in service availability and scheduling by introducing structured booking management.

The system also aims to ensure secure access through authentication and role-based control, allowing customers, service providers, and administrators to operate within clearly defined boundaries. Improving transparency and communication between users is another key objective, particularly through real-time updates on booking status.

Finally, the platform is designed with scalability in mind, so that it can be extended to other regions in the future with minimal structural changes. These objectives helped maintain focus throughout development and served as reference points during system testing and evaluation.

1.3 Scope of the Project

The scope of MH26 Services is limited to managing household and personal services within a defined geographical region. The platform focuses on core service coordination features rather than attempting to cover all possible functionalities at once. This controlled scope helped ensure system stability and usability.

Within this scope depicted by **Fig 1.1**, the system supports user registration, service provider onboarding, service booking, and booking status tracking. Customers can browse available services, select providers, and monitor service progress. Service providers can manage their profiles, respond to booking requests, and update service status during execution. Administrative users are responsible for verifying providers and overseeing platform activity.

The project does not include full-scale online payment processing in its current version. Instead, emphasis is placed on reliable service coordination and workflow management. Features such as live location tracking and payment gateway integration are considered outside the present scope and are intended for future enhancement.

By clearly defining these boundaries early in development, unnecessary complexity was avoided. This approach allowed the system to be evaluated effectively while leaving room for gradual expansion based on future requirements.

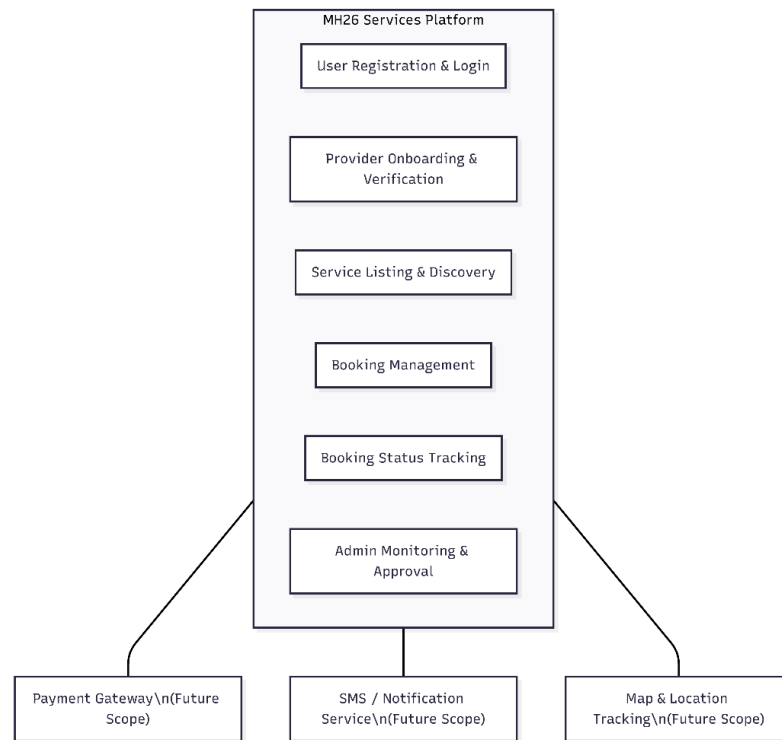


Fig. 1.1: Scope of MH26 Services Platform

1.4 Applications of the Project

The MH26 Services platform can be applied in various residential, commercial, and community-based environments where local service coordination is required. For households, the system provides a convenient way to locate verified service professionals for both routine maintenance and emergency situations, reducing the uncertainty involved in finding reliable help.

Small businesses can also use the platform to access dependable service providers without spending time on repeated searches or informal coordination. From an employment perspective, the system creates opportunities for local professionals to reach a wider customer base. By maintaining records of completed services, providers can gradually build credibility and receive more consistent work.

The platform can also be adapted for organized environments such as housing societies, small offices, and community-managed facilities. In these settings, the system helps standardize service requests and improves coordination by maintaining clear service records.

Due to its modular design, MH26 Services can be customized for deployment in other cities with minimal changes. While local data may differ, the underlying workflow remains consistent, making the platform suitable for gradual regional expansion.

1.5 Overview of the Project

MH26 Services is implemented as a web-based application that follows a modular and layered architecture. We designed the system so that the frontend handles user interaction through a simple and intuitive interface, while the backend manages business logic, data validation, and communication with the database.

Key user actions such as registration, booking creation, and service completion follow predefined workflows to ensure consistency and prevent invalid operations. Real-time mechanisms are used to keep users informed about booking updates and status changes, reducing delays in communication between customers and service providers.

Security is integrated throughout the system using authentication, authorization, and request validation. These measures ensure that users can access only the features relevant to their roles and that sensitive data is protected. Overall, the system architecture is designed to balance simplicity, reliability, and scalability while remaining suitable for real-world use in a local environment.

A high-level overview of the system workflow is illustrated below in **Fig 1.2**.

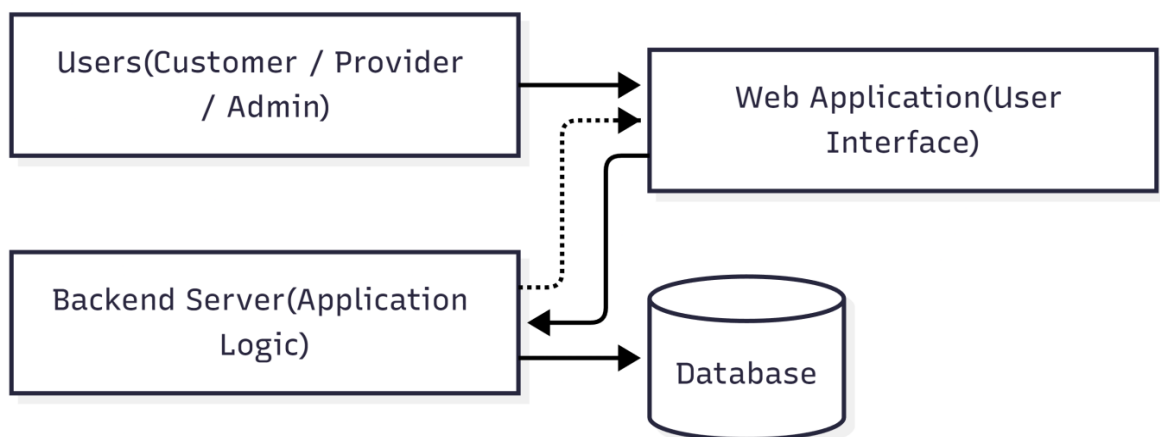


Fig. 1.2: High-Level System Overview Diagram

User Role	Core Responsibilities
Customer	Service discovery, booking requests, status tracking
Provider	Profile management, booking handling, service completion
Admin	Provider verification, system monitoring, access control

Table 1.1: User Roles and Core Responsibilities

The project overview provides a holistic understanding of how different components of the system interact. This high-level perspective helps readers visualize system behavior before examining detailed analysis and design aspects discussed in subsequent chapters.

System Analysis and Design

System analysis and design for the MH26 Services platform focused on understanding how local service booking actually happens in practice and identifying where the process fails. Instead of assuming an ideal workflow, we studied the existing manual approach used by customers and service providers and mapped its limitations. This helped ensure that the system design addressed real problems rather than theoretical ones.

During analysis, we observed that service requests are typically managed through phone calls, personal references, and informal coordination. This leads to missed calls, unclear scheduling, and lack of accountability. Based on these observations, the design phase focused on creating a structured, secure, and scalable digital system that simplifies service coordination while remaining easy to use for local users. This chapter explains how requirements were identified and how system behavior, data flow, and architecture were designed for MH26 Services.

2.1 Requirement Analysis

Requirement analysis for MH26 Services began with examining the challenges faced by both customers and service providers in the existing unorganized system. Customers often rely on saved phone numbers or local references to arrange services, which is unreliable and time-consuming. There is no clear way to track service requests, confirm availability, or ensure accountability.

From the service provider's perspective, work opportunities depend heavily on informal networks or contractors. There is no structured way to manage incoming requests or maintain service history. These observations made it clear that a centralized platform was required to connect both parties efficiently.

Based on this understanding, several core requirements were identified. The system needed to support secure user registration, structured booking management, and timely communication between users. Trust and accountability were also critical requirements, which led to the inclusion of provider verification and controlled access mechanisms.

In addition to functional needs, practical constraints were considered during analysis. The platform had to be simple enough for users with limited digital familiarity, while still being capable of handling multiple bookings and users simultaneously. These requirements guided all major design decisions and ensured alignment with real-world usage conditions.

2.2 Functional Requirements

The functional requirements define what actions the MH26 Services system must support in order to operate effectively. These requirements were identified based on user interactions observed during requirement analysis.

The system allows users to register and authenticate securely before accessing any services. Different roles—customers, service providers, and administrators—are handled through role-based login to ensure controlled access. Customers can browse service categories, view provider profiles, and create booking requests by selecting suitable dates and times.

To avoid scheduling conflicts, the system validates booking requests before confirmation. Service providers are notified when new booking requests are created and can accept or reject them based on availability. Providers can also update service status during and after job completion.

Administrative users are responsible for verifying service providers and monitoring system activity. The system maintains records of users, providers, and bookings to support tracking and evaluation.

These functionalities ensure smooth interaction between users and reliable operation of the platform. Clearly defining these operations also provides a solid base for implementation and testing.

2.3 Non-Functional Requirements

Non-functional requirements describe how well the system should perform rather than what it should do. For MH26 Services, these requirements were defined based on expected real-world usage.

From a usability perspective, the system is designed to be easy to use, allowing users to complete common actions such as booking creation within a few simple steps. The interface is intended to work across common screen sizes and modern web browsers.

Performance requirements focus on responsiveness. Actions such as login and booking creation are expected to complete within a few seconds under normal load. Real-time notifications should be delivered without noticeable delay to keep users informed.

Security requirements ensure that authentication data and booking information are protected. Unauthorized access attempts must be blocked, and sensitive data should only be accessible to authorized users. Reliability is addressed by ensuring stable operation and graceful error handling without system crashes.

Scalability and maintainability were also considered. The system is designed to support growth in users and bookings without major architectural changes. Modular design ensures that future updates or fixes can be implemented without disrupting existing functionality.

2.4 Use Case Analysis

Use case analysis was carried out to understand how different users interact with the system. MH26 Services involves three primary actors: Customer, Service Provider, and Administrator. Each actor interacts with the system based on specific responsibilities.

Customers use the system to register, search for services, create bookings, and track service progress. Service providers interact with the platform to manage profiles, respond to booking requests, and complete assigned jobs. Administrators oversee provider verification, monitor activity, and manage access when required.

Identifying these interactions early helped ensure that all required system functionalities were included. Use case analysis also helped define clear boundaries between user roles, reducing ambiguity during implementation. The use case diagram is illustrated below in **Fig 2.1**.

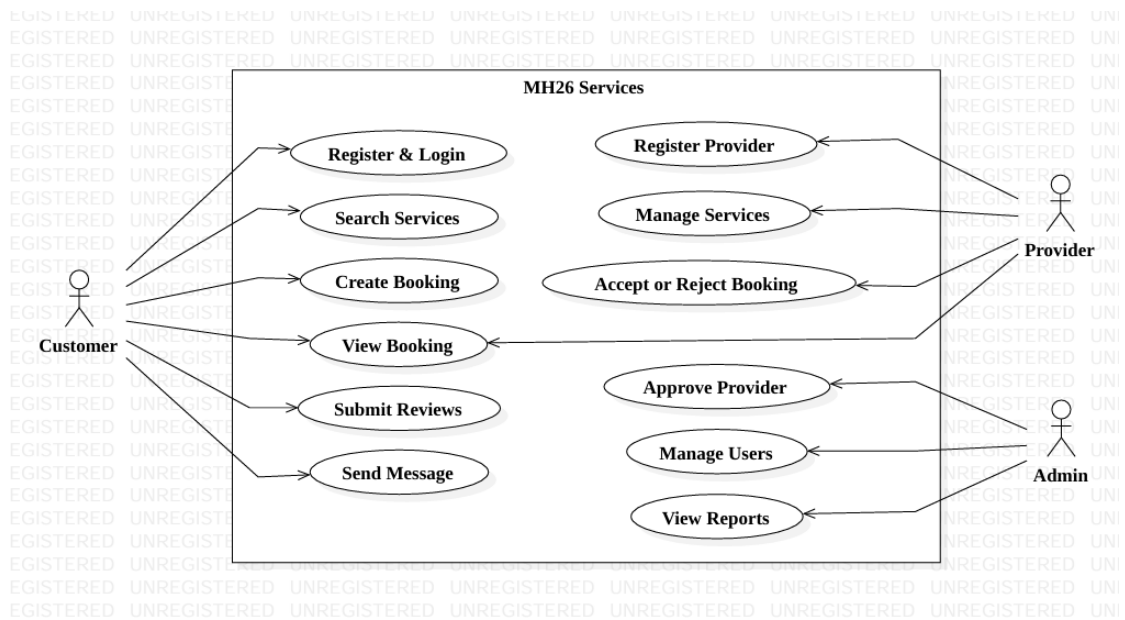


Fig. 2.1: Use Case Diagram of MH26 Services

2.5 Data Flow Analysis

Data flow analysis was used to understand how information moves through the system during different operations. In MH26 Services, data flows from users through the web interface to the backend for processing and storage.

For example, when a customer creates a booking request, the input data is sent to the backend where availability checks and validations are performed. Once processed, the booking information is stored in the database and relevant status updates are sent to users. Similar flows occur during provider acceptance and service completion.

Analyzing these flows helped identify potential bottlenecks and security concerns early. This ensured that data is processed efficiently while maintaining consistency and integrity throughout system operations.

By careful analysis of the flow, we have drafted a diagram given below, in **Fig 2.2** to show a proper flow of system.

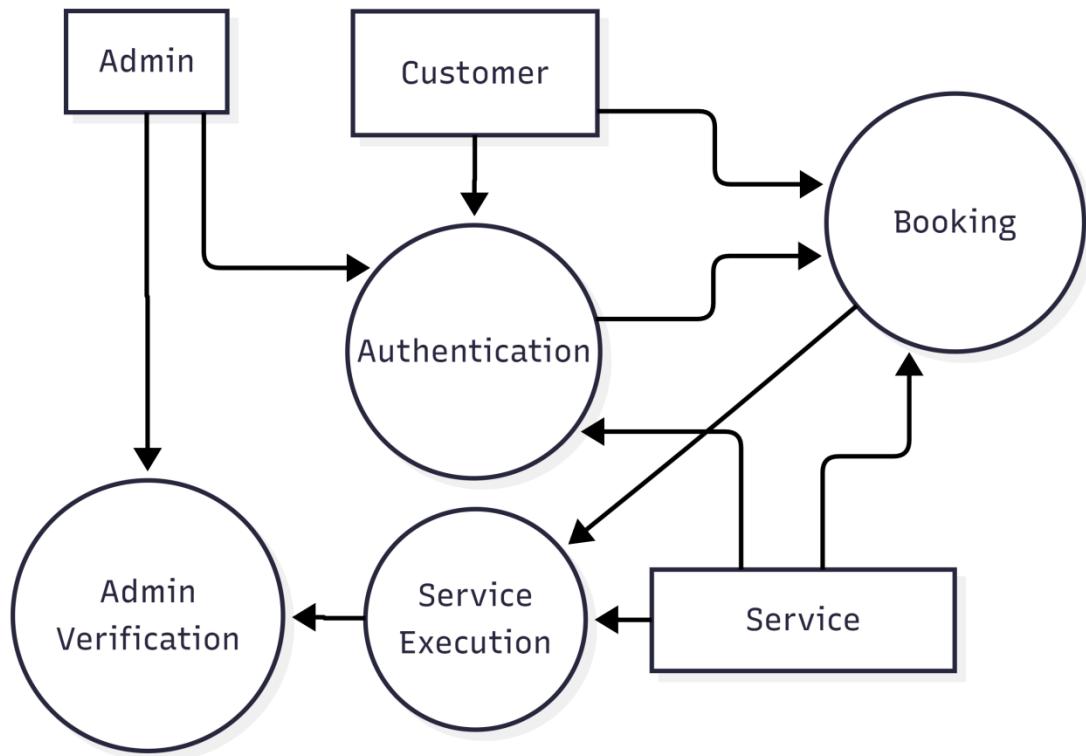


Fig. 2.2: Data Flow Diagram of Booking Process

2.6 Database Design

The database design for MH26 Services is based on a relational model to manage structured application data. Key entities include users, service providers, and bookings, each representing real-world objects.

Users and providers are linked to bookings through unique identifiers, ensuring accurate association. The booking entity stores scheduling and status information, allowing reliable tracking of service requests. Normalization principles were applied to minimize redundancy and improve data consistency.

A well-structured database supports efficient querying and simplifies future enhancements. This design ensures that system data remains organized as the platform grows.

Given below in **Fig 2.3** is ER Diagram for MH26 Services system, designed carefully with proper cardinalities and relation.

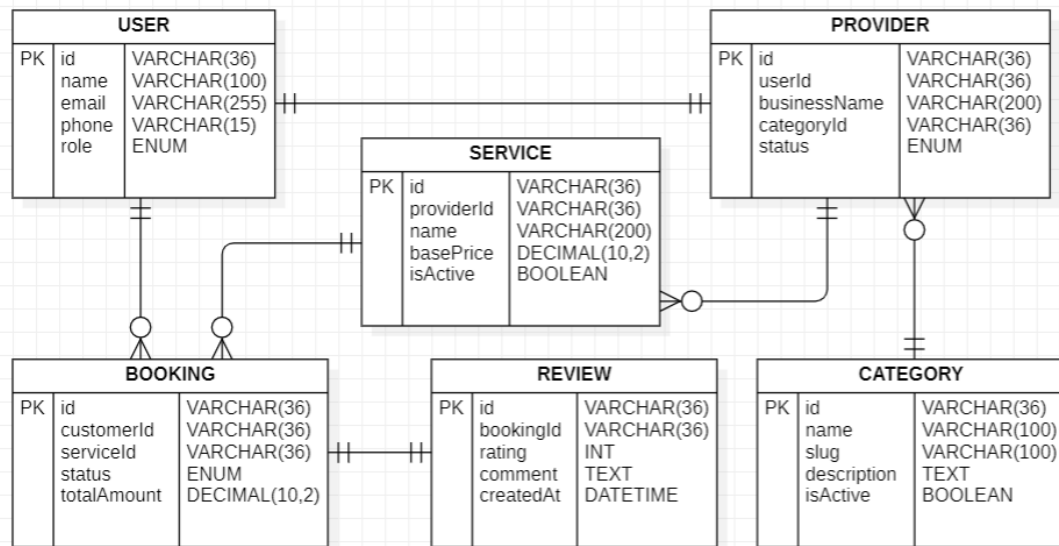


Fig. 2.3: ER Diagram Overview

2.7 User Roles and Access Rights

User roles and access rights define what actions each user can perform within the system. MH26 Services implements role-based access control to prevent unauthorized operations.

Customers can browse services and manage their bookings. Service providers can access only the bookings assigned to them and update service status. Administrators have higher privileges that allow them to verify providers and monitor system activity.

This separation of responsibilities improves accountability and reduces misuse. It also simplifies system management by clearly defining permissions for each role.

2.8 System Architecture

The system architecture of MH26 Services follows a layered approach consisting of a presentation layer, application logic layer, and data layer. Users interact with the system through a web-based interface that communicates with backend services using structured requests.

The backend layer processes business logic and handles communication with the database. This separation improves modularity and allows individual components to

be updated with minimal impact on others. The architecture supports scalability, maintainability, and long-term flexibility.

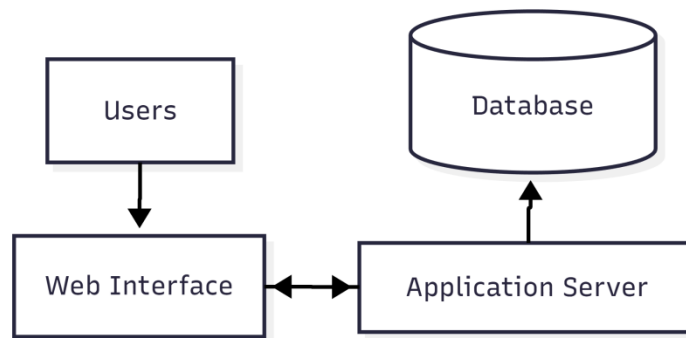


Fig. 2.4: High-Level System Architecture

The system architecture of MH26 Services follows a layered design approach that separates user interaction, application logic, and data storage. Users interact with the system through a web-based interface, which forwards requests to the application server for processing. The application server applies business rules and communicates with the database for persistent data storage. This architectural separation improves maintainability, scalability, and overall system reliability.

System Implementation

The system implementation phase involved converting the analyzed requirements and system design of MH26 Services into a working software application. Instead of treating implementation as a single step, we developed the system in modular parts and integrated them gradually to ensure stability and clarity. This approach allowed us to identify issues early and refine individual components without affecting the entire system.

The implementation follows a layered architecture where each layer is responsible for a specific task. This separation helped simplify testing, improve maintainability, and ensure that future enhancements can be added without major restructuring. This chapter explains how different system components were implemented and integrated to support reliable service booking and management.

3.1 Technology Stack Overview

MH26 Services is implemented using a modern full-stack web development stack selected based on project requirements and practical constraints. The technologies were chosen to support secure authentication, structured data management, and real-time interaction between users.

The frontend stack focuses on usability and responsiveness, ensuring smooth interaction for different user roles. The backend stack handles validation, business logic, and communication with the database. Persistent data storage is managed through a relational database to maintain consistency and integrity.

All the technology stacks are mentioned in **Table 3.1**, with **Fig 3.1** showing their logo.

Layer	Technology	Purpose
Frontend	React	User interface development
Backend	Node.js, Express	API handling and logic processing
Database	PostgreSQL	Relational data storage
ORM	Prisma	Database abstraction
Styling	Tailwind CSS	Responsive UI design
Real-Time	Socket.io	Live notifications

Table 3.1: Technology Stack Used



Fig. 3.1: Technology Stack Overview

3.1.1 Frontend – React

React was used to develop the frontend of MH26 Services due to its component-based structure and efficient rendering behavior. The interface was divided into reusable components such as authentication forms, service listings, booking modals, and dashboards. This improved consistency across the application and simplified future updates.

The frontend dynamically updates based on user actions and responses received from the backend. This ensures that users receive immediate feedback without unnecessary page reloads, resulting in a smoother experience.

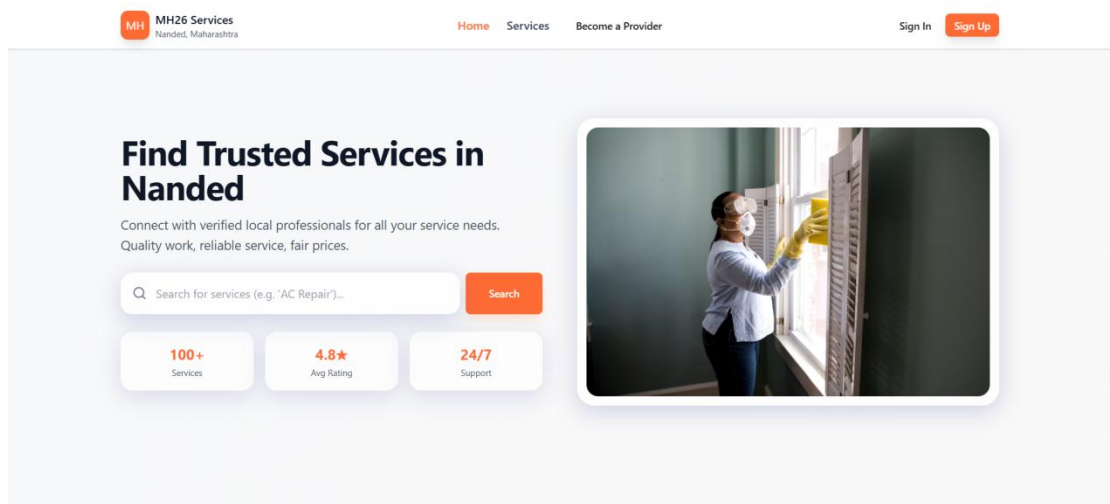


Fig. 3.1(a): Customer Home Page and Service Listing (React UI)

3.1.2 Backend – Node.js and Express

Node.js was used as the backend runtime environment because of its non-blocking and event-driven nature, which is suitable for handling multiple requests efficiently. Express was used as the web framework to define API routes and manage middleware.

The backend processes core business logic such as authentication, booking validation, service status updates, and invoice generation. To maintain clarity, the backend code is organized into routes, controllers, services, and middleware. This separation ensures that request handling remains independent from core logic.

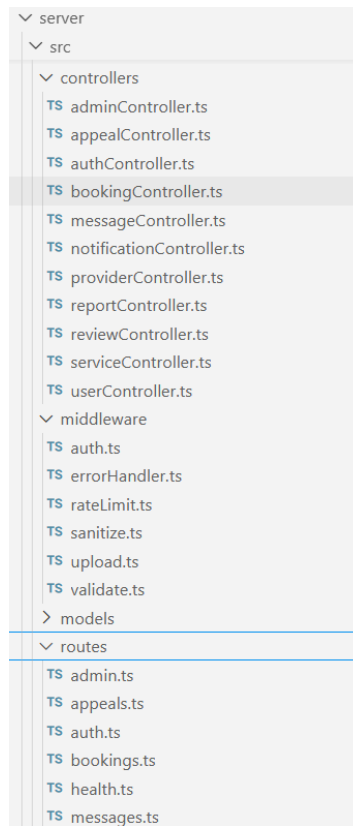


Fig. 3.1(b): Backend API Structure and Routing

3.1.3 Database – PostgreSQL

PostgreSQL is used as the relational database for storing application data. It was selected for its reliability, strong consistency, and support for relational integrity, which are important for managing bookings and user relationships.

The database stores structured information related to users, service providers, bookings, and financial records. This structure ensures that data remains consistent and scalable as the system grows.

Filters None Fields All Showing 25 of 25 Add record								
☐	id	name	email	phone	passwordHash	avatarUrl	role	emailVerified
☐	0d3011cb-77f4-412d-8d0b...	Iran Khan	jai.hind.furniture@gmail...	+91-9922334455	\$2b\$12\$7b1gC0g3J496ETg...	null	PROVIDER	true
☐	152369e7-9b8b-44ce-b7a1...	Sirran Kaur	signate.beauty@gmail.com	+91-9876509877	\$2b\$12\$Qu1b/AMuC.2cn3qp...	null	PROVIDER	true
☐	16549d76-17b7-4016-bde0...	Rajesh Sharma	tirupati.electronics@gm...	+91-9890123456	\$2b\$12\$Uq/9OAL04A3r1VPL...	null	PROVIDER	true
☐	176f97a1-0d68-48cc-943f...	Siddhi Manager	siddhi.mangal@gmail.com	+91-9890567891	\$2b\$12\$E+EX2tvtptkD6Gmqd...	null	PROVIDER	true
☐	188b1e07-2a07-420a-83ec...	Admin User	admin@h26services.com	+91-9000000000	\$2b\$12\$aa2an7HC/VXEP0za...	null	ADMIN	true
☐	1d3f4a20-4e27-4891-accf...	Shri Omer	shri.plumbing@gmail.com	+91-9823456789	\$2b\$12\$M7Todes1A0K3kVP/F...	null	PROVIDER	true
☐	242caf7d-fb73-4e54-b652...	Anil Sharma	anil.electrical@gmail...	+91-9823456004	\$2b\$12\$1hMZ8i/nU85IoTIK...	null	PROVIDER	true
☐	2db27614-0c3e-4d20-822d...	Rama Omer	rama.sales@gmail.com	+91-9823456019	\$2b\$12\$B/SFieVkmVTxrRGf...	null	PROVIDER	true
☐	44d4a518-3336-454a-a531...	Santosh Jadhav	super.sanitary@gmail.com	+91-9823456003	\$2b\$12\$96D3NBL0VfgjjytS...	null	PROVIDER	true
☐	44f5e1ac-6ffb-4b43-b8ad...	Dutta Darshdeep	gamerdani322@gmail.com	8432822104	\$2b\$12\$sv3cyc81wbvAT6...	null	CUSTOMER	true
☐	4db8b510-8063-4e81-b3a4...	Rajesh Joshi	joshi.caterers@gmail.com	+91-9823456011	\$2b\$12\$Jm20TB2c.fikfND...	null	PROVIDER	true
☐	5058b531-ff3b-400c-becb...	Vishwakarma Rao	vishwakarma.furniture@g...	+91-9823456017	\$2b\$12\$E8n6QZIXasevMyp...	null	PROVIDER	true
☐	5e498a4f-f609-419e-a19a...	Oasis Omer	oasis.beauty@gmail.com	+91-9823456014	\$2b\$12\$dZ1SKD09Y8a1V7/c...	null	PROVIDER	true
☐	60cc0f02-3990-461a-8f61...	Ganesh Shinde	ekdant.enterprises@gmai...	+91-8877665544	\$2b\$12\$R5D0rhmo7thDzE...	null	PROVIDER	true
☐	7c09d6a4-8afa-43bd-903c...	Rahul Udgire	samarth.plumbing@gmail...	+91-9823456001	\$2b\$12\$Mh.s6y.BT1x1nqfC...	null	PROVIDER	true
☐	82506b53-b210-46ff-905e...	Nitin Jadhav	jai.malhar.caters@gmail...	+91-9823567890	\$2b\$12\$Xj31CUpICQ07eC1...	null	PROVIDER	true
☐	92043d4e-861d-424a-ac2e...	Shri Sahayog	sahayog.seva@gmail.com	+91-9823456012	\$2b\$12\$HKhq8RigfI.Ctun...	null	PROVIDER	true
☐	a01c9926-c235-40bc-a612...	Krishna Sharma	krishna.furniture@gmail...	+91-9823456018	\$2b\$12\$SQhyRk1ptDeIXCa...	null	PROVIDER	true
☐	a7f23ce2-9f43-4fcb-a09d...	Sameer Khan	sameer.plumbers@gmail.c...	+91-9876543210	\$2b\$12\$FHu5YqEPKam84B...	null	PROVIDER	true

Fig. 3.1(c): Database Tables and Relationships

3.1.4 ORM – Prisma

Prisma ORM is used to manage interactions between the backend application and the database. Database models are defined using a structured schema, allowing queries to be written in a type-safe and readable manner.

Using Prisma reduced manual SQL handling and minimized the risk of common database errors. It also ensured synchronization between application models and the underlying database schema throughout development.

3.1.5 Styling – Tailwind CSS

Tailwind CSS was used to style the frontend components of the application. Its utility-first approach allowed styles to be applied directly through predefined classes, reducing the need for separate CSS files.

Tailwind also made it easier to implement responsive layouts, ensuring that the interface adapts well to different screen sizes and devices.

3.1.6 Real-Time Communication – Socket.io

Socket.io was integrated to enable real-time communication between users and the system. It is used to deliver instant notifications for events such as booking creation, acceptance, and completion.

This approach eliminates delays caused by manual refresh or polling and significantly improves coordination between customers and service providers.

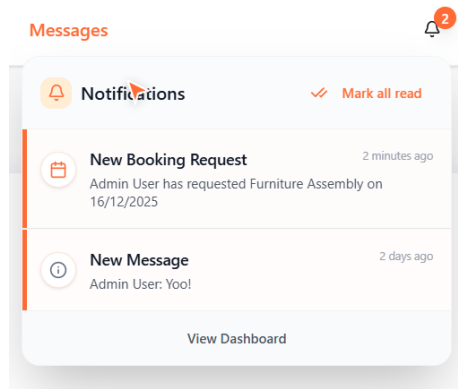


Fig. 3.1(d): Real-Time Booking Notification

3.2 Development Approach

While working on the MH26 Services project, we did not follow a fixed step-by-step development plan from the beginning. As students, many requirements became clear only after we started building the system and experimenting with different features. Because of this, we naturally followed an Agile-style approach during development, even if it was not strictly planned at the start.

Initially, our focus was only on getting the basic idea working, such as user registration, login, and understanding how customers and service providers would interact with the system. Once this basic flow was implemented, we realized several improvements were needed in both functionality and user experience. Features like booking validation, service status tracking, and OTP-based service completion were added gradually as we gained more clarity.

Development was done in small parts rather than building everything at once. After completing one feature, we tested it, discussed issues among ourselves, and then moved to the next part. In many cases, we had to revisit earlier modules to fix logic gaps or improve validation. This iterative way of working helped us avoid major errors and made the system more stable over time.

Following this flexible development approach allowed us to learn from mistakes, improve our design decisions, and continuously refine the system. Overall, working in an Agile manner made the project more manageable and helped us understand how real-world software development adapts to changing requirements.

3.3 Frontend Implementation

The frontend is implemented using a modular component-based design. Separate components handle authentication, booking creation, provider dashboards, and administrative views. Routing logic ensures that users are directed to the appropriate interface based on their role.

Client-side validation is applied to forms to detect invalid inputs early, improving usability and reducing unnecessary backend requests. Session and state management techniques are used to maintain user login status and update data dynamically.

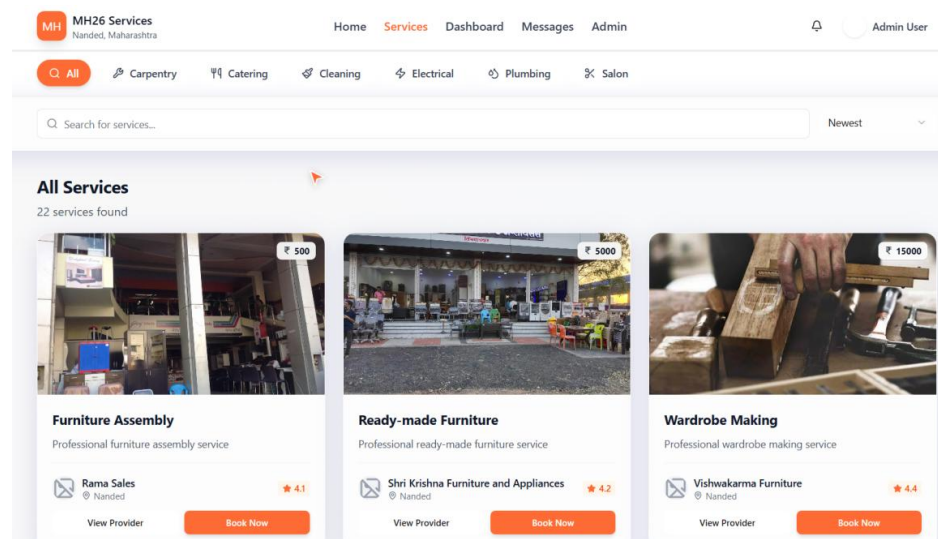


Fig. 3.2: Service Listing Screen

The 'Book Service' modal is displayed. It features a header 'Book Service'. Below it, the selected service 'Furniture Assembly' is shown with a price tag of ₹500. The form contains several input fields: 'DATE' with a calendar icon and 'dd-mm-yyyy' format, 'TIME SLOT' with a clock icon and 'Select Time' text, 'SERVICE ADDRESS' with a location pin icon and 'House no., Street, Area' text, 'CITY' with 'Nanded' entered, and 'PINCODE' which is empty. There is also a 'SPECIAL REQUIREMENTS (OPTIONAL)' section with a text area containing the placeholder 'Describe specific needs or issues...'. At the bottom, there are two buttons: 'Cancel' and 'Confirm Booking'.

Fig. 3.3: Booking Modal Interface

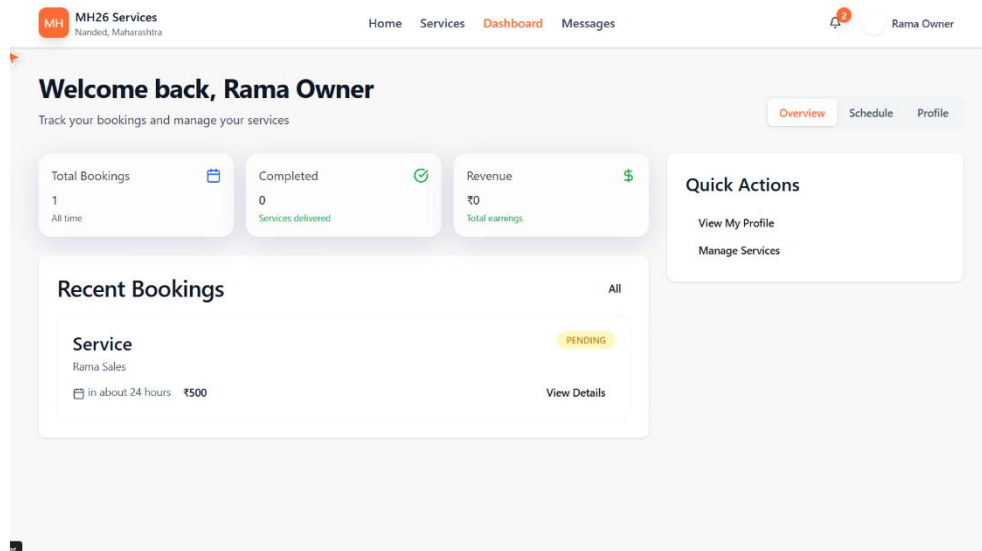


Fig. 3.4: Provider Dashboard View

3.4 Backend Implementation

The backend serves as the core processing layer of MH26 Services. It handles API requests, applies business rules, and communicates with the database.

Middleware is used to authenticate users, validate requests, and protect sensitive routes. Error-handling mechanisms ensure that invalid requests do not crash the system and that meaningful error responses are returned.

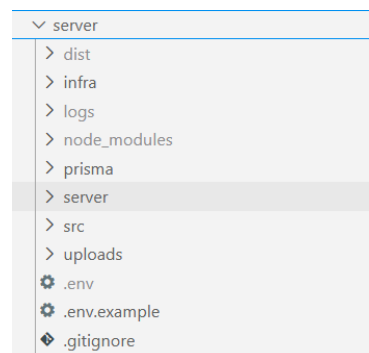


Fig. 3.5: Backend Project Structure

3.5 Database Implementation

The database schema is designed to reflect real-world relationships between system entities. Prisma models define tables for users, service providers, and bookings, ensuring proper association through foreign keys.

Entity	Description
User	Stores customer and admin data
Provider	Stores provider profile and status
Booking	Stores booking details and lifecycle

Table 3.2: Core Database Entities

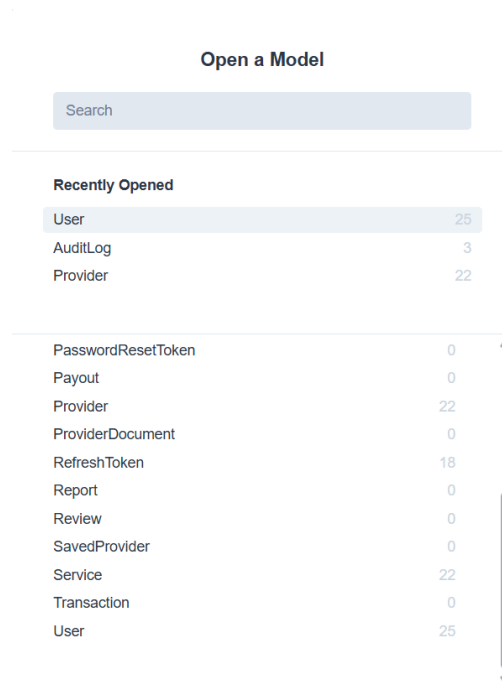


Fig. 3.6: Database Schema Implementation

3.6 Authentication and Authorization Implementation

Authentication ensures that only verified users can access the system. Users must provide valid credentials before performing any protected actions. Role-based authorization restricts access based on user type.

Customers, service providers, and administrators are assigned different permissions to prevent unauthorized access and maintain system security. Session handling ensures secure access during active usage.

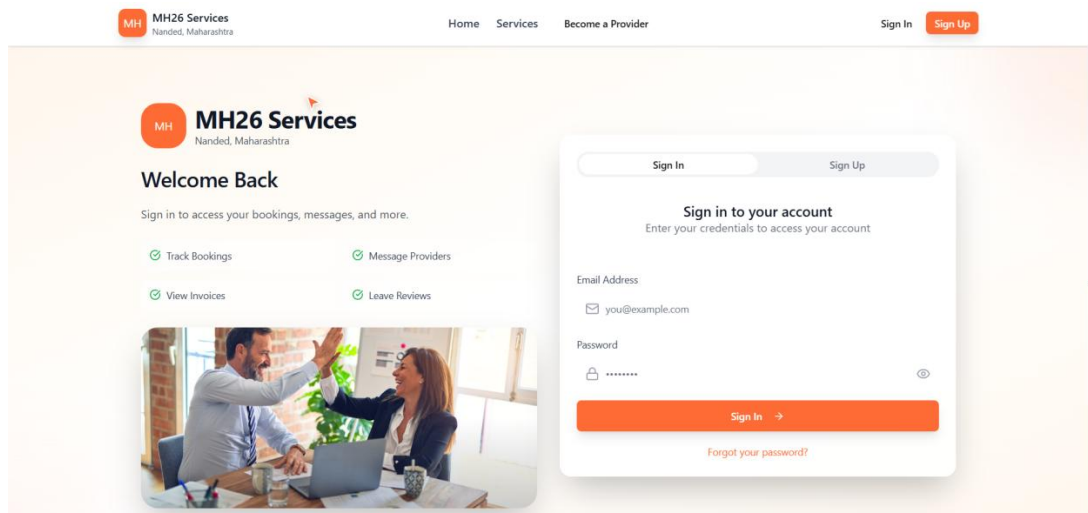


Fig. 3.7: Login and Registration Interface

3.7 Booking Management Implementation

Booking management forms the core functionality of MH26 Services. The system manages the entire booking lifecycle, from request creation to service completion.

Before confirming a booking, provider availability is checked to prevent scheduling conflicts. Booking status transitions are controlled to maintain consistency, and users are notified at each important stage.

3.8 Real-Time Communication Implementation

Real-time notifications are sent when booking events occur, such as request creation, acceptance, or completion. This keeps users informed without requiring manual refresh.

This feature improves transparency and reduces miscommunication between customers and service providers.

3.9 Financial and Invoice System Implementation

A mock financial module is implemented to simulate real-world transaction handling. It tracks customer spending, provider earnings, and platform commission for each completed service.

Invoices are generated dynamically and displayed through the frontend, providing users with clear transaction records. The system design allows future integration with real payment gateways.

Component	Description
Customer Spending	Total service cost
Provider Earnings	Net earnings
Platform Fee	Commission per booking

Table 3.3: Financial Components Tracked

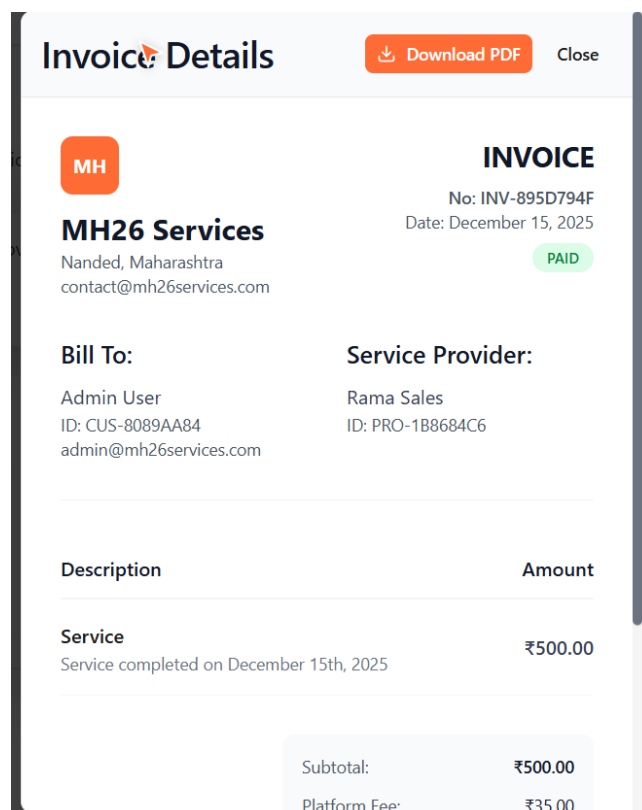


Fig. 3.8: Invoice Preview Screen

3.10 Summary

This chapter explained the implementation of the MH26 Services platform, covering technology selection, frontend and backend development, database integration, and core business workflows. The modular implementation approach ensured maintainability, scalability, and clear separation of responsibilities. Key functionalities such as authentication, booking management, real-time notifications, and invoice generation were implemented successfully. The completed system was then prepared for testing and evaluation, which are discussed in the next chapter.

Results and System Evaluation

This chapter discusses the results obtained after testing and evaluating the MH26 Services platform. The purpose of testing was to verify whether the implemented system behaves as expected under practical usage conditions. Instead of limiting evaluation to theoretical correctness, testing focused on real user interactions, system stability, and response behavior.

After completing system implementation, testing was carried out in multiple stages to validate functionality, security, performance, and overall system reliability. Both individual modules and complete workflows were tested to ensure that the system performs consistently when different components interact. The following sections describe the testing approach adopted and the results observed during evaluation.

4.1 Testing

Testing was conducted using a combination of manual testing and controlled test scenarios. Each module was tested independently before testing integrated workflows. This helped identify issues early and ensured that changes in one module did not affect others unexpectedly.

4.1.1 Functional Testing

Functional testing was performed to verify that all major features of the system operate according to the intended design. Core functionalities such as user authentication, service booking, OTP-based completion, and invoice generation were tested using both valid and invalid inputs.

During testing, user actions were simulated from different roles to ensure correct system behavior. Invalid inputs were intentionally provided to confirm that the system handles errors gracefully without crashing.

Test Case ID	Module	Test Description	Expected Result	Result
FT-01	Authentication	Login with valid credentials	Successful login	Pass
FT-02	Authentication	Login with incorrect password	Access denied	Pass
FT-03	Booking	Create booking with valid date and time	Booking created	Pass
FT-04	Booking	Attempt booking with past date	Booking rejected	Pass
FT-05	Booking	Overlapping booking for same provider	Booking blocked	Pass
FT-06	Completion	Enter correct OTP	Booking completed	Pass
FT-07	Completion	Enter incorrect OTP	Completion denied	Pass
FT-08	Invoice	Generate invoice after completion	Invoice displayed	Pass

Table 4.1: Functional Testing Test Cases

The results indicate that all core functionalities behave as expected and align with the defined system requirements.

4.1.2 Integration Testing

Integration testing focused on verifying communication between frontend components, backend services, and the database. Complete user workflows were tested to ensure data flows correctly across system layers.

Special attention was given to booking creation and completion workflows, where multiple modules interact simultaneously.

Test Case ID	Integrated Modules	Test Scenario	Observed Result
IT-01	Frontend + Backend	Booking request from user interface	Request processed correctly
IT-02	Backend + Database	Save booking data	Data stored accurately
IT-03	Backend + Socket	New booking notification	Provider notified instantly
IT-04	Completion + Finance	OTP verification and invoice update	Financial data updated

Table 4.2: Integration Testing Test Cases

4.1.3 Validation Testing

Validation testing was conducted to ensure that incorrect, incomplete, or unexpected inputs are handled properly. Both client-side and server-side validations were tested to prevent invalid data from being processed.

Test Case ID	Input Type	Test Condition	Result
VT-01	Registration	Required fields left empty	Error message displayed
VT-02	Booking	Time selection omitted	Booking blocked
VT-03	OTP	Non-numeric OTP entered	Verification failed
VT-04	Form Input	Excessive character input	Input restricted

Table 4.3: Input Validation Test Cases

4.1.4 Security Testing

Security testing was performed to ensure that unauthorized users cannot access restricted areas of the system. Role-based access control and OTP verification were tested extensively.

Test Case ID	Security Aspect	Test Scenario	Expected Result	Status
ST-01	Authorization	Customer accessing admin panel	Access denied	Pass
ST-02	Authorization	Provider accessing customer-only route	Access denied	Pass
ST-03	OTP Security	Provider attempts completion without OTP	Operation blocked	Pass
ST-04	Session	Access protected route without login	Redirected to login	Pass

Table 4.4: Security Testing Test Cases

4.1.5 Performance and Deployment Testing

Performance testing was conducted under normal usage conditions to observe system responsiveness. Deployment testing was also performed after hosting the application to identify any runtime issues.

Test Area	Observation	Result
Login Response	Completed within 3 seconds	Pass
Booking Creation	Completed within 3 seconds	Pass
Notification Delivery	Instant delivery	Pass
Backend Cold Start	Initial delay observed	Acceptable
Database Connectivity	Stable during testing	Pass

Table 4.5: Performance and Deployment Testing

Minor delays during backend cold starts were observed on free-tier hosting, which is acceptable for demonstration purposes.

4.2 User Interface Output

The user interface was evaluated across different user roles to assess usability and consistency. The system correctly displays role-specific dashboards and restricts access based on user permissions.

Customers were able to browse services, create bookings, and track status without difficulty. Service providers could manage booking requests and update service progress. The administrative interface provided an overview of provider verification and system activity.

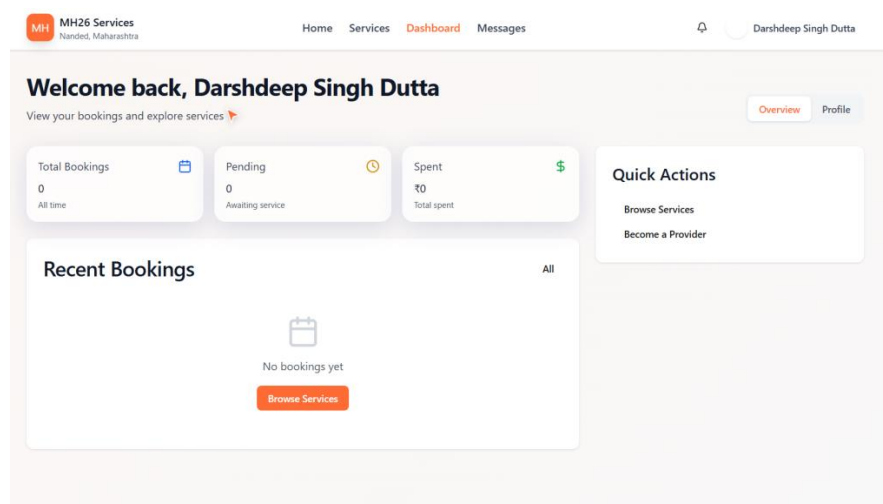


Fig. 4.1: Customer Home Page Output

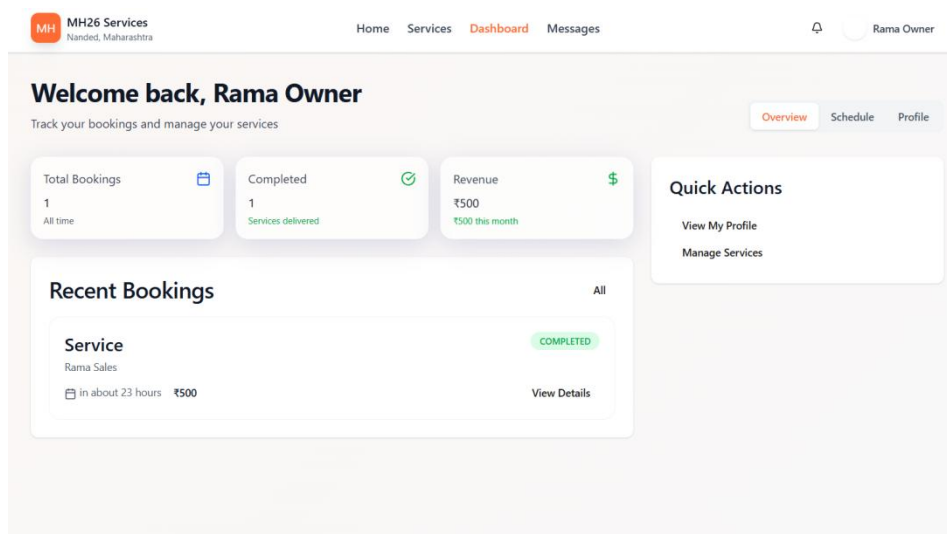


Fig. 4.2: Provider Dashboard Output

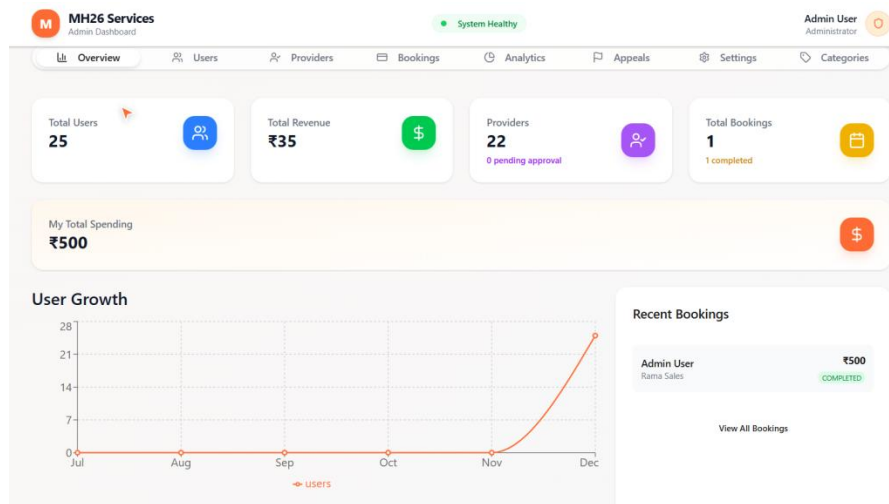


Fig. 4.3: Admin Panel Output

4.3 Authentication and Access Control Results

Authentication testing confirmed that only registered users can access system features. Incorrect credentials are rejected, and unauthorized access attempts are blocked.

Role-based routing ensures that users are redirected to the correct dashboard after login. This confirms that access control is consistently enforced across the platform.

4.4 Booking Creation and Validation Results

Multiple booking scenarios were tested using different date and time inputs. The system successfully prevented bookings with invalid or past dates. Provider availability checks ensured that overlapping bookings were not permitted.

Successful booking creation resulted in immediate status updates and notifications, confirming correct booking workflow execution.

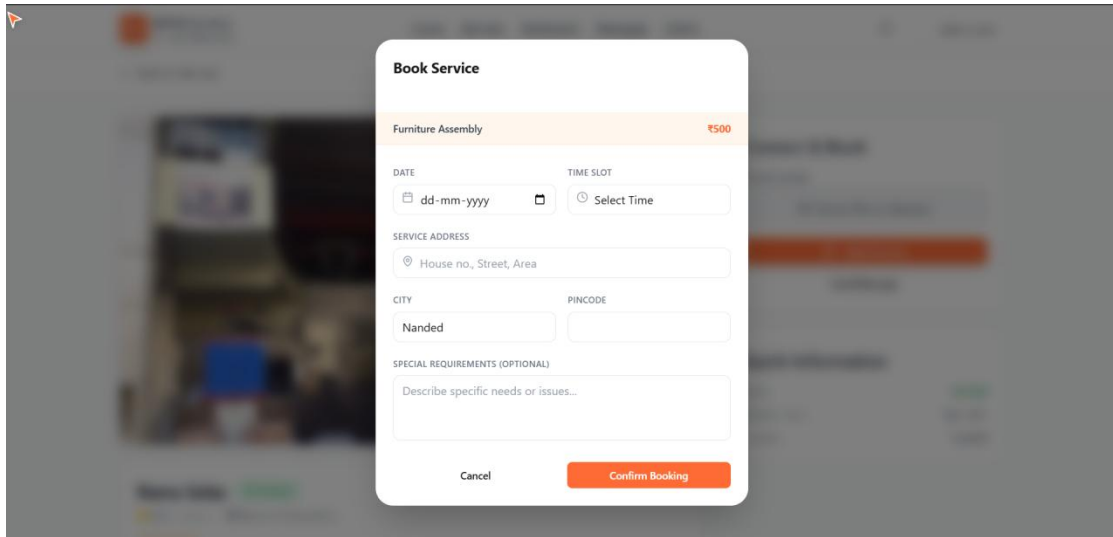


Fig. 4.4: Booking Creation Output Screen

4.5 Real-Time Notification Results

Real-time notifications were tested by triggering booking-related events such as request creation, acceptance, and completion. Notifications were delivered instantly without requiring page refresh.

This confirms effective real-time communication between system users.

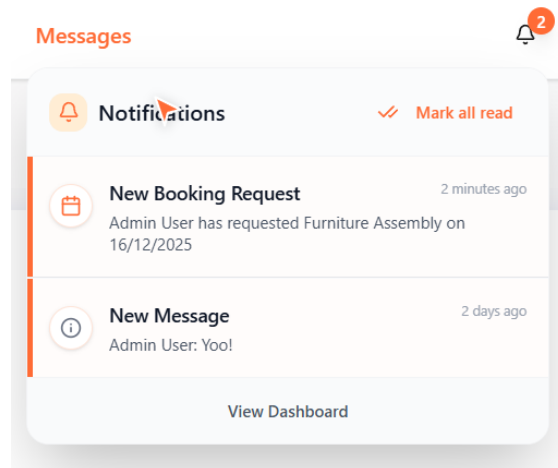


Fig. 4.5: Real-Time Booking Notification Output

4.6 Service Completion and OTP Verification Results

The OTP-based service completion process was tested to verify secure service validation. OTPs were generated only at the completion stage and delivered exclusively to customers.

Service completion was allowed only after correct OTP verification. Financial updates were processed only after successful verification, preventing unauthorized completion attempts.

4.7 Invoice Generation and Financial Output

After service completion, invoices were generated automatically. Each invoice included service details, charges, platform fees, and the total amount.

Invoices were displayed in a clear and structured format, allowing users to review transaction details easily.

Invoice Details	
MH MH26 Services Nanded, Maharashtra contact@mh26services.com	INVOICE No: INV-895D794F Date: December 15, 2025 PAID
Bill To: Admin User ID: CUS-8089AA84 admin@mh26services.com	Service Provider: Rama Sales ID: PRO-1B8684C6
Description	Amount
Service Service completed on December 15th, 2025	₹500.00
Subtotal: ₹500.00	
Platform Fee: ₹35.00	

Fig. 4.6: Invoice Preview Output

4.8 System Performance Observation

The system was observed under regular usage conditions involving multiple user interactions. No system crashes were encountered during testing, and response times remained within acceptable limits.

The backend handled concurrent operations reliably, indicating stable server-side processing and database interaction.

4.9 Summary of Testing and Results

The testing and evaluation of MH26 Services confirm that the system performs reliably under practical conditions. All core functionalities were validated through functional, integration, security, and performance testing. The observed stability and responsiveness indicate that the platform is suitable for demonstration and small to medium-scale deployment.

Challenges and Future Scope

This chapter reflects on the overall development of the MH26 Services platform by discussing the features implemented, challenges faced during development, lessons learned, and possible future improvements. Unlike previous chapters that focus on design and implementation, this chapter evaluates the project from a practical and real-world perspective, considering system limitations, ethical aspects, and long-term sustainability.

5.1 Key Features Implemented

MH26 Services implements a complete workflow for managing hyperlocal service bookings in a structured digital environment. The system supports user registration and role-based access for customers, service providers, and administrators, ensuring that each user interacts with the platform within defined responsibilities.

A key feature of the system is the end-to-end booking lifecycle management. This includes booking creation by customers, acceptance or rejection by service providers, service execution, and completion verification. Real-time notifications help keep all parties informed at each stage of the process.

The OTP-based service completion mechanism plays an important role in ensuring trust and accountability. Service completion and financial updates are allowed only after successful OTP verification. The system also includes invoice generation and financial tracking to simulate real-world transaction handling, providing transparency to users.

5.2 Challenges Faced

Several challenges were encountered during the development of MH26 Services. One of the main challenges was handling booking concurrency and preventing scheduling conflicts. Designing logic that accurately checks provider availability without impacting performance required multiple iterations and careful testing.

Another significant challenge was implementing secure OTP-based service completion. Ensuring that OTPs were generated, delivered, and verified correctly—

while preventing misuse—required repeated testing and refinement. Small errors in this flow could lead to incorrect service completion or security issues.

Deployment also introduced challenges due to the use of free-tier hosting services. Issues such as backend cold starts, service idling, and limited email functionality affected real-time behavior. These limitations required adjustments in testing expectations and reinforced the importance of choosing suitable infrastructure for production deployment.

5.3 Learning Outcomes

The development of MH26 Services provided valuable learning experiences across technical and practical domains. The project strengthened understanding of full-stack development, including frontend component design, backend architecture, and relational database management.

Practical exposure was gained in implementing authentication systems, role-based access control, and real-time communication using sockets. The project also improved debugging and problem-solving skills, particularly when dealing with deployment-related issues and unexpected runtime behavior.

Beyond technical learning, the project highlighted the importance of designing systems based on real-world constraints rather than ideal assumptions.

5.4 Future Enhancements

Although the current version of MH26 Services fulfills its primary objectives, several enhancements can be introduced in future versions. Integration of a real payment gateway would allow actual financial transactions instead of simulated wallet-based calculations.

SMS-based OTP delivery can be added to improve accessibility for users who may not regularly use email. Additional features such as live service tracking, mobile application support, and subscription-based plans for service providers could further enhance usability and scalability.

Expanding the platform to support multiple cities with region-specific configurations is another important future goal.

5.5 Real-World Use and Deployment

MH26 Services is designed to be applicable in real-world environments, particularly in semi-urban areas where service coordination is largely unorganized. The system can be deployed for local communities, housing societies, or small businesses that require a structured approach to service management.

With appropriate upgrades to hosting infrastructure and communication services, the platform can transition from a demonstration system to a production-ready application. The modular architecture supports such upgrades with minimal changes to the existing system.

5.6 System Limitations

Despite its strengths, the system has certain limitations. The current deployment relies on free-tier hosting services, which can cause delays due to service idling. Email-based OTP delivery may not be ideal for users with limited internet access or delayed email notifications.

The system currently focuses on core service management features and does not include advanced analytics, automated dispute resolution, or recommendation mechanisms. These limitations define clear areas for improvement in future versions.

5.7 Ethical Considerations and Data Privacy

Ethical considerations were addressed throughout system design and implementation. User data is collected and accessed only when necessary for system functionality. Role-based access control ensures that users cannot view or modify data beyond their permitted scope.

Sensitive operations such as service completion and financial updates require verification to prevent misuse. These measures help maintain user trust and encourage responsible system usage.

5.8 Performance Evaluation and Testing Results

System performance was evaluated under normal usage conditions involving multiple user interactions. Core operations such as login, booking creation, and notification delivery were completed within acceptable response times.

The system remained stable during testing, with no critical failures observed. These results indicate that the platform can handle typical usage scenarios effectively at a small to medium scale.

5.9 Cost Analysis and Resource Requirements

The project was developed primarily using free and open-source technologies, which helped minimize development costs. Free-tier cloud services were used for hosting, making the system cost-effective for demonstration and academic evaluation.

For real-world deployment, moderate monthly expenses would be required to ensure reliable hosting, uninterrupted services, and proper communication infrastructure. These costs are justified by the platform's potential practical value.

5.10 Sustainability and Environmental Impact

MH26 Services indirectly supports sustainability by reducing unnecessary travel and manual coordination. Digital service booking minimizes paperwork and helps optimize service scheduling, which can reduce fuel usage and time wastage.

Cloud-based deployment further reduces the need for physical infrastructure, contributing to more efficient resource utilization.

5.11 Summary

This chapter evaluated the MH26 Services platform beyond technical implementation by examining challenges, learning outcomes, limitations, and future potential. The project demonstrates how modern web technologies can be applied to solve real-world problems in local service coordination. While the current system meets its intended objectives, it also provides a strong foundation for further development and real-world deployment.

Conclusion

The MH26 Services project demonstrates how a focused, hyperlocal software solution can address practical problems in organizing local service coordination. By concentrating on real-world service interactions, the system provides a structured platform for service discovery, booking management, and service verification while keeping the user experience simple and accessible. Features such as role-based access control, real-time notifications, and OTP-based service completion were implemented to improve transparency, accountability, and trust throughout the service lifecycle.

The project also highlights the effective use of modern web technologies to solve everyday community-level problems rather than large-scale commercial use cases. Although the current system has limitations related to free-tier deployment and simulated financial transactions, it successfully meets its intended objectives and performs reliably under practical conditions. Overall, MH26 Services serves as a functional prototype with strong potential for future enhancement and real-world deployment with appropriate infrastructure upgrades.

References

- [1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed., New York: McGraw-Hill Education, 2015, pp. 1–720.
- [2] I. Sommerville, *Software Engineering*, 10th ed., Boston: Pearson Education, 2016, pp. 1–816.
- [3] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed., New York: McGraw-Hill Education, 2011, pp. 1–1376.
- [4] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed., Upper Saddle River, NJ: Pearson Education, 2007, pp. 1–686.
- [5] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000, pp. 1–180.
- [6] T. Erl, “Service-oriented architecture: Concepts, technology, and design,” *IEEE Computer Society*, vol. 38, no. 11, Nov. 2005, pp. 34–41.